Accessing hardware devices



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



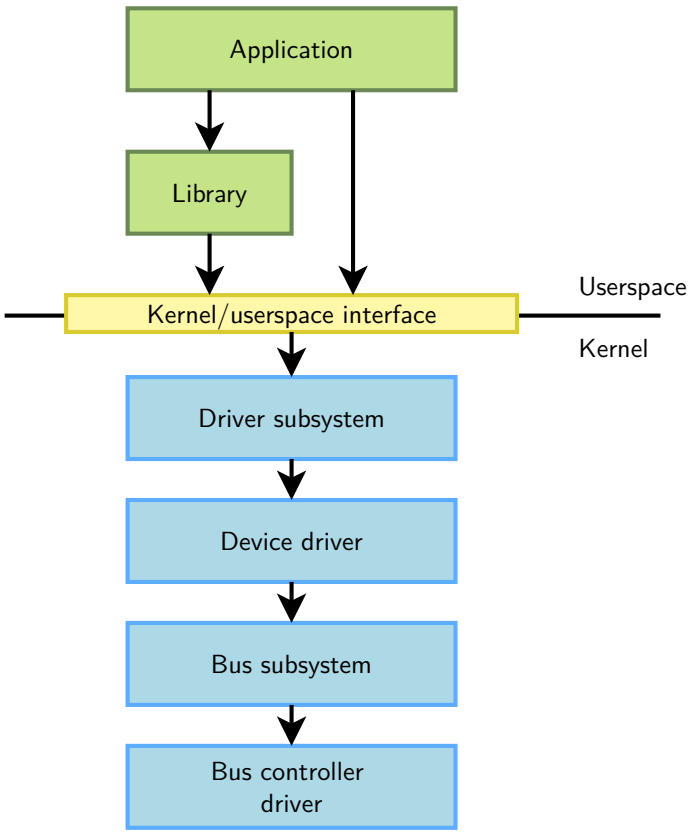
Kernel drivers



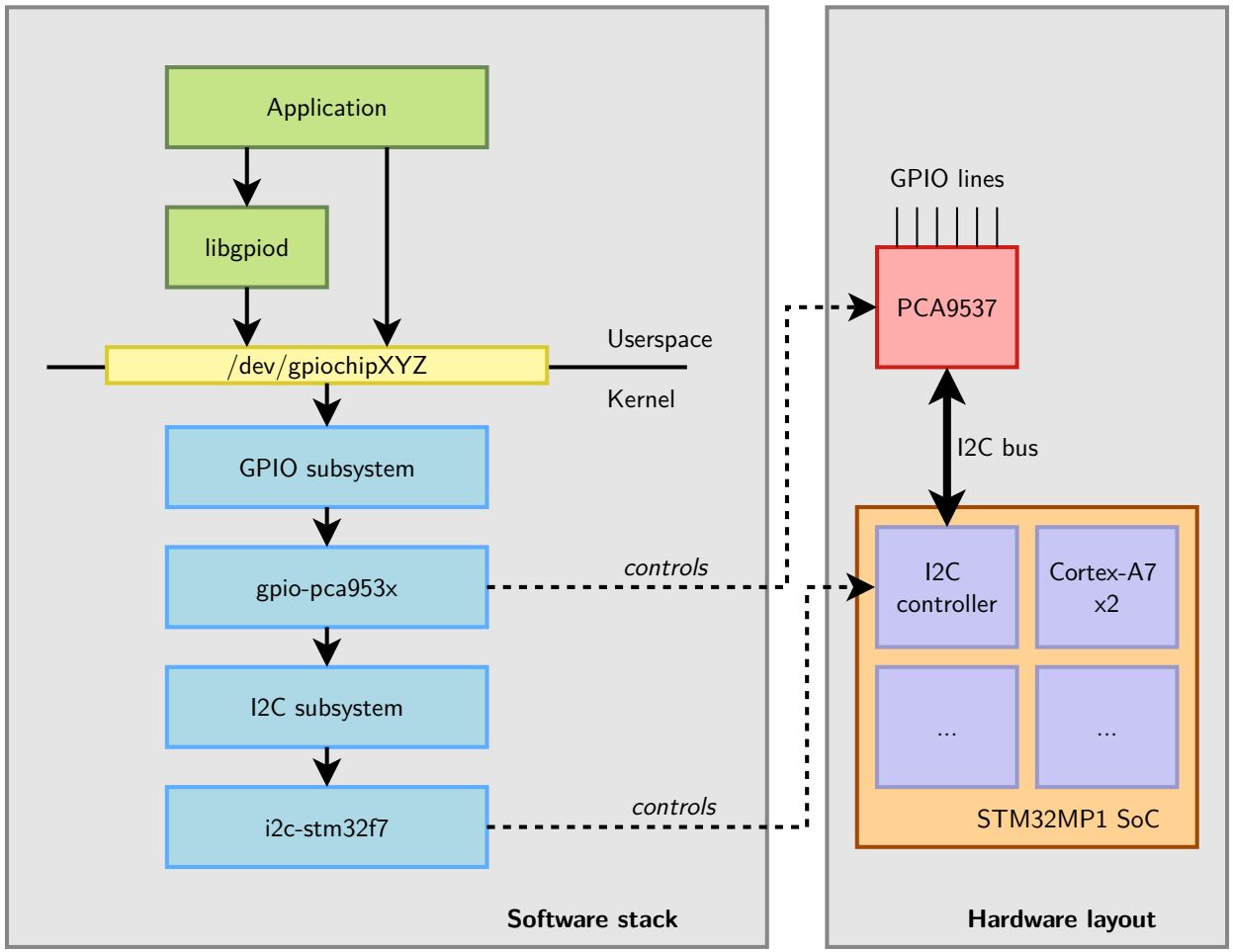
Typical software stack for hardware access

From the bottom to the top:

- A *bus controller driver* in the kernel drives an I2C, SPI, USB, PCI controller
- A *bus subsystem* provides an API for drivers to access a particular type of bus: I2C, SPI, PCI, USB, etc.
- A *device driver* in the kernel drives a particular device connected to a given bus
- A *driver subsystem* exposes features of certain class of devices, through a standard *kernel/user-space interface*
- An application can access the device through this standard *kernel/user-space interface* either directly or through a library.



Stack illustrated with a GPIO expander





Standardized user-space interface

- Strong advantage of kernel drivers: they expose a standard *kernel to user-space interface*
- All devices of the same class (e.g GPIO controllers) will expose the same *kernel to user-space interface*
- Applications don't have to know the details of the GPIO controller, they just need to know the standard user-space interface valid for all GPIO controllers
- Applications can use existing open-source libraries that leverage this standard user-space interface
- Such kernel drivers can also be used internally inside the kernel, for example if one driver needs to control a GPIO directly (reset signal, interrupt signal, etc.)



Numerous kernel subsystems for device classes

- Networking stack for Ethernet, WiFi, CAN, 802.15.4, etc.
- GPIO
- Video4Linux for camera, video encoders/ decoders
- DRM for display controllers, GPU
- ALSA for audio
- IIO for ADC, DAC, gyroscopes, sensors, and more
- MTD for flash memory
- PWM
- Input for keyboard, mouse, touchscreen, joystick
- Watchdog
- RTC for real-time clocks
- remoteproc for auxiliary processors
- crypto for cryptographic accelerators
- hwmon for hardware monitoring sensors
- block layer for block storage

and many more

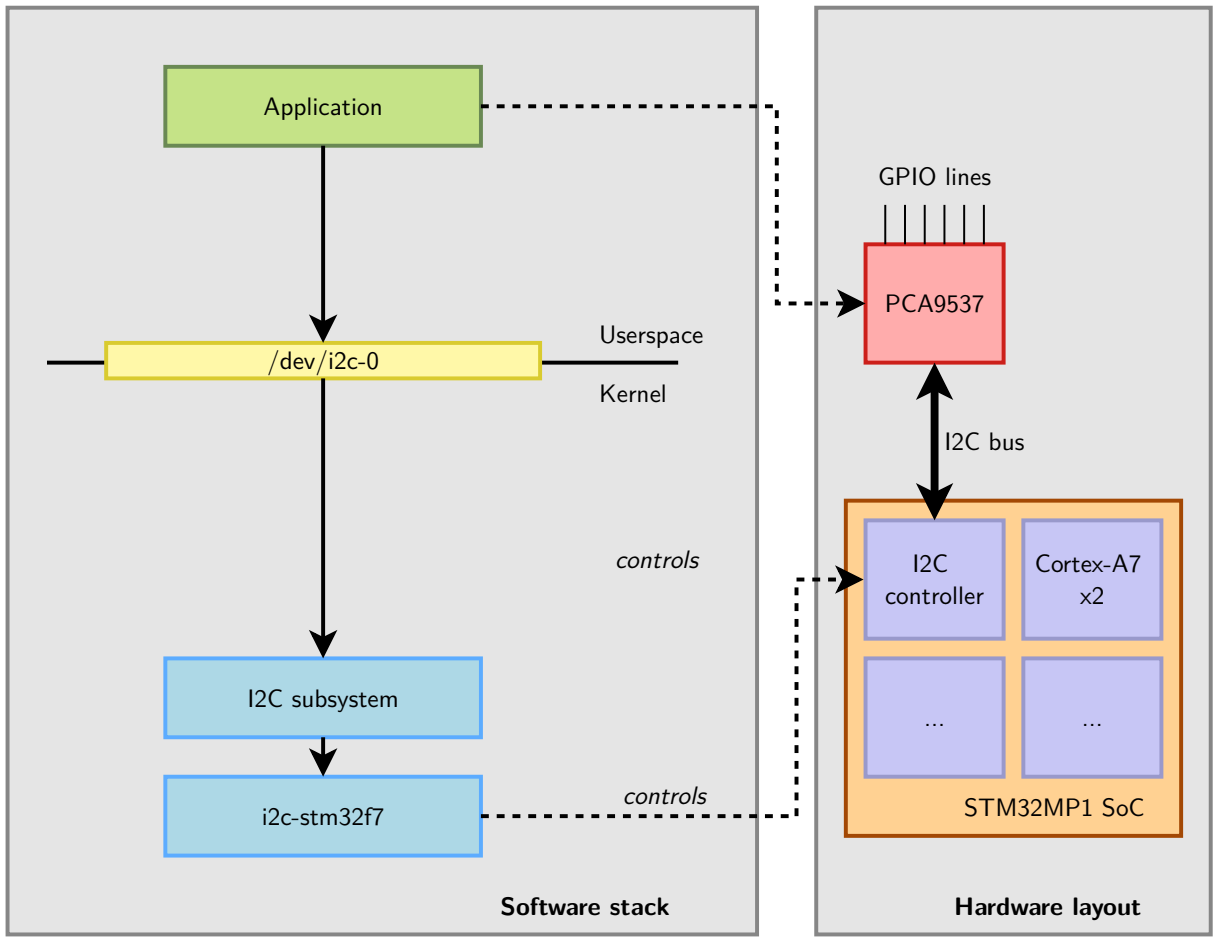


Accessing devices directly from user-space

- Even though device drivers in the kernel are preferred, it is also possible to access devices directly from user-space
- Especially useful for very specific devices that do not fit in any existing kernel subsystems
- The kernel provides the following mechanisms, depending on the bus:
  - I2C: i2c-dev
  - SPI: spidev
  - Memory-mapped: UIO
  - USB: `/dev/bus/usb`, through libusb
  - PCI: sysfs entries for PCI

Accessing devices directly from user-space: GPIO example

This diagram shows what's not recommended to do → for a GPIO controller, a kernel driver is preferred





What can go wrong with a user-space driver?

- You write your GPIO driver in user-space: other kernel drivers cannot use GPIOs from this GPIO controller
  - Other devices that use GPIO signals from this controller for reset, interrupt, etc. cannot control/configure those signals
  - Your application is less portable: it will take many changes to support another type of GPIO controller.
- You write your touchscreen driver in user-space: the standard Linux graphics stack components cannot use your touchscreen
- You write your network driver in user-space
  - You can probably send/receive packets
  - But you cannot leverage the Linux kernel networking stack for IP, TCP, UDP, etc.
  - And none of the Linux networking applications can use your network device



Upstream drivers vs. out-of-tree drivers

- The *upstream* Linux kernel contains thousands of drivers
  - This is the best place to look for drivers
  - Drivers have been reviewed and approved by the community
  - They comply with standard interfaces
- Vendor kernels often include additional drivers, directly in the kernel tree
- Device vendors sometimes also provide *out of tree drivers*
  - Their source code is provided separately from the Linux kernel tree
  - Quality is often dubious
  - Compatibility issues when updating to newer kernel releases
  - Not always use standard user-space interfaces
  - Example: https://github.com/lwfinger/rtl8723ds
  - Avoid them when possible!



Finding Linux kernel drivers

- `grep` in the Linux kernel tree is your *best friend*
  - For I2C, SPI and memory-mapped devices, matching of the driver is done based on the device name → *grep* for variants of the device name and vendor
  - For USB, PCI, matching is done either on the vendor ID/product ID, or the class → *grep* for these
- Driver file names are sometimes named in a "generic" way, not necessarily reflecting all devices they support.
  - Example: drivers/gpio/gpio-pca953x.c supports much more than just PCA953x. See the full list of devices supported by this driver



Finding Linux kernel drivers: an example

- You have a Maxim Integrated MAX7313 GPIO expander on I2C
- Search in the Linux kernel

```
$ git grep -i max7313
drivers/gpio/gpio-pca953x.c:    { "max7313", 16 | PCA953X_TYPE | PCA_INT, },
drivers/gpio/gpio-pca953x.c:    { .compatible = "maxim,max7313", .data = OF_953X(16, PCA_INT), },
```

- drivers/gpio/gpio-pca953x.c seems to support it
- Read drivers/gpio/Makefile to learn which kernel configuration option enables this driver:

```
obj-$(CONFIG_GPIO_PCA953X)                  += gpio-pca953x.o
```

- Conclusion: you need to enable CONFIG_GPIO_PCA953X in your kernel configuration



User-space interfaces to drivers



User-space interfaces for hardware devices

For a high-level perspective: three main interfaces to access hardware devices exposed by the Linux kernel

- Device nodes in `/dev`
- Entries in the *sysfs* filesystem
- Network sockets and related APIs



Devices in */dev/*

- One of the kernel important roles is to **allow applications to access hardware devices**
- In the Linux kernel, most devices are presented to user space applications through two different abstractions
  - **Character** device
  - **Block** device
- Internally, the kernel identifies each device by a triplet of information
  - **Type** (character or block)
  - **Major** (typically the category of device)
  - **Minor** (typically the identifier of the device)
- See Documentation/admin-guide/devices.txt for the official list of reserved type/major/minor numbers.



Block vs. character devices

- Block devices
  - A device composed of fixed-sized blocks, that can be read and written to store data
  - Used for hard disks, USB keys, SD cards, etc.
- Character devices
  - Originally, an infinite stream of bytes, with no beginning, no end, no size. The pure example: a serial port.
  - Used for serial ports, terminals, but also sound cards, video acquisition devices, frame buffers
  - Most of the devices that are not block devices are represented as character devices by the Linux kernel



Devices: everything is a file

- A very important UNIX design decision was to represent most *system objects* as files
- It allows applications to manipulate all *system objects* with the normal file API (`open`, `read`, `write`, `close`, etc.)
- So, devices had to be represented as files to the applications
- This is done through a special artifact called a **device file**
- It is a special type of file, that associates a file name visible to user space applications to the triplet *(type, major, minor)* that the kernel understands
- All *device files* are by convention stored in the `/dev` directory



Device files examples

Example of device files in a Linux system

```
$ ls -l /dev/ttyS0 /dev/tty1 /dev/sda /dev/sda1 /dev/sda2 /dev/sdc1 /dev/zero
brw-rw---- 1 root disk    8,  0 2011-05-27 08:56 /dev/sda
brw-rw---- 1 root disk    8,  1 2011-05-27 08:56 /dev/sda1
brw-rw---- 1 root disk    8,  2 2011-05-27 08:56 /dev/sda2
brw-rw---- 1 root disk    8, 32 2011-05-27 08:56 /dev/sdc
crw------- 1 root root    4,  1 2011-05-27 08:57 /dev/tty1
crw-rw---- 1 root dialout 4, 64 2011-05-27 08:56 /dev/ttyS0
crw-rw-rw- 1 root root    1,  5 2011-05-27 08:56 /dev/zero
```

Example C code that uses the usual file API to write data to a serial port

```c
int fd;
fd = open("/dev/ttyS0", O_RDWR);
write(fd, "Hello", 5);
close(fd);
```



Creating device files

- Before Linux 2.6.32, on basic Linux systems, the device files had to be created manually using the `mknod` command
  - `mknod /dev/<device> [c|b] major minor`
  - Needs root privileges
  - Coherency between device files and devices handled by the kernel was left to the system developer
- The `devtmpfs` virtual filesystem can be mounted on `/dev` → the kernel automatically creates/removes device files
  - CONFIG_DEVTMPFS_MOUNT → asks the kernel to mount *devtmpfs* automatically at boot time (except when booting on an initramfs).



Better handling of device files: *udev* and *mdev*

- *devtmpfs* is great, but its capabilities are limited, so complementary solutions exist
- **udev**
  - daemon that receives events from the kernel about devices appearing/disappearing
  - can create/remove device files (but that's done by *devtmpfs* now), adjust permission/ownership, load kernel modules automatically, create symbolic links to devices
  - according to rules files in `/lib/udev/rules.d` and `/etc/udev/rules.d`
  - used in almost all desktop Linux distributions
  - https://en.wikipedia.org/wiki/Udev
- **mdev**
  - lightweight implementation of *udev*, part of BusyBox
  - https://wiki.gentoo.org/wiki/Mdev



Examples of user-space interfaces in `/dev`

- Serial-ports: `/dev/ttyS*`, `/dev/ttyUSB*`, `/dev/ttyACM*`, etc.
- GPIO controllers (modern interface): `/dev/gpiochipX`
- Block storage devices: `/dev/sd*`, `/dev/mmcblk*`, `/dev/nvme*`
- Flash storage devices: `/dev/mtd*`
- Display controllers and GPUs: `/dev/dri/*`
- Audio devices: `/dev/snd/*`
- Camera devices: `/dev/video*`
- Watchdog devices: `/dev/watchdog*`
- Input devices: `/dev/input/*`
- and many more...



sysfs filesystem

- `block/`, symlinks to all block devices, in `/sys/devices`
- `bus/`, one sub-folder by type of bus
- `class/`, one sub-folder per class (category of devices): input, leds, pwm, etc.
- `dev/`
  - `block/`, one symlink per block device, named after major/minor
  - `char/`, one symlink per character device, named after major/minor
- `devices/`, all devices in the system, organized in a slightly chaotic way, see this article
- `firmware/`, representation of firmware data
  - `devicetree/`, directory and file representation of Device Tree nodes and properties
- `fs/`, properties related to filesystem drivers
- `kernel/`, properties related to various kernel subsystems
- `module/`, properties about kernel modules
- `power/`, power-management related properties



sysfs filesystem example

- `/sys/bus/i2c/drivers`: all device drivers for devices connected on I2C busses

```
[...]
edt_ft5x06
stpmic1
[...]
```

- `/sys/bus/i2c/devices`: all devices in the system connected to I2C busses

```
0-002a -> ../../../devices/platform/soc/40012000.i2c/i2c-0/0-002a
0-0039 -> ../../../devices/platform/soc/40012000.i2c/i2c-0/0-0039
0-004a -> ../../../devices/platform/soc/40012000.i2c/i2c-0/0-004a
1-0028 -> ../../../devices/platform/soc/5c002000.i2c/i2c-1/1-0028
1-0033 -> ../../../devices/platform/soc/5c002000.i2c/i2c-1/1-0033
i2c-0 -> ../../../devices/platform/soc/40012000.i2c/i2c-0
i2c-1 -> ../../../devices/platform/soc/5c002000.i2c/i2c-1
i2c-2 -> ../../../devices/platform/soc/40012000.i2c/i2c-0/i2c-2
```



sysfs filesystem example

/sys/bus/i2c/devices/0-002a/

```
lrwxrwxrwx    driver -> ../../../../../../bus/i2c/drivers/edt_ft5x06
-rw-r--r--    gain
drwxr-xr-x    input
-r--r--r--    modalias
-r--r--r--    name
lrwxrwxrwx    of_node -> ../../../../../../firmware/devicetree/base/soc/i2c@40012000/touchscreen@2a
-rw-r--r--    offset
-rw-r--r--    offset_x
-rw-r--r--    offset_y
drwxr-xr-x    power
-rw-r--r--    report_rate
lrwxrwxrwx    subsystem -> ../../../../../../bus/i2c
-rw-r--r--    threshold
-rw-r--r--    uevent
```

- `driver`, symlink to the driver directory in `/sys/bus/i2c/drivers`
- `of_node`, symlink to the directory for the Device Tree node describing this device



Example of driver interfaces in *sysfs*

- All devices are visible in *sysfs*, whether they have an interface in `/dev` or not
  - Usually `/dev` is to access the device
  - `/sys` is more about properties of the devices
- However, some devices only have a *sysfs* interface
  - LED: `/sys/class/leds`, see documentation
  - PWM: `/sys/class/pwm`, see documentation
  - IIO: `/sys/bus/iio`, see documentation
  - etc.



Accessing GPIOs

A class of devices worth mentioning is GPIOs (*General Purpose Input Output*)

- The GPIOs can be accessed through a legacy interface in `/sys/class/gpios`
  - You will find many instructions on the Internet about how to drive GPIOs through this interface.
  - However, this interface is deprecated and has multiple shortcomings:
    - GPIOs remain exported if the process using them crashes
    - Need to compute the GPIO numbers, such numbers are not stable
- A new interface recommended: libgpiod
  - Based on `/dev/gpiochipx` character devices
  - Implementing advanced features not possible with the legacy interface
  - Of course, this is a C library
  - But it also provides command line utilities: `gpiodetect`, `gpioset`, `gpioget`...
  - The only constraint is to cross-compile them for your target (the legacy interface could be used without any additional software).



Other virtual filesystems

- *debugfs*
  - Conventionally mounted in `/sys/kernel/debug`
  - Contains lots of debug information from the kernel, including device related
  - `/sys/kernel/debug/pinctrl` for pin-mux debugging, `/sys/kernel/debug/gpio` for GPIO debugging, `/sys/kernel/debug/pwm` for PWM debugging, etc.
  - https://www.kernel.org/doc/html/latest/filesystems/debugfs.html
- *configfs*
  - Conventionally mounted in `/sys/kernel/config`
  - Allows to manage configuration of advanced kernel mechanisms
  - Example: configuration of USB gadget functionalities
  - Documentation/filesystems/configfs.rst



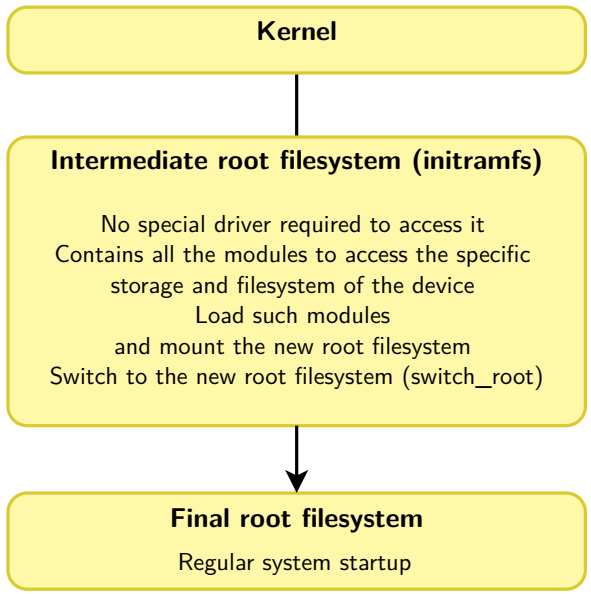
Using kernel modules



Why kernel modules?

- Primary reason: keep the kernel image minimal, and load drivers on-demand depending on the hardware detected
  - Needed to create a generic kernel configuration that works on many platforms
  - Used by all desktop/server Linux distributions
- But also useful for
  - Driver development: allows to modify, build and test a driver without rebooting
  - Boot time reduction: allows to defer the initialization of a driver after user-space has started critical applications

Using kernel modules to support many different devices and setups



The modules in the initramfs are updated every time a kernel upgrade is available.



Module installation and metadata

- As discussed earlier, modules are installed in `/lib/modules/<kernel-version>/`
- Compiled kernel modules are stored in `.ko` (*Kernel Object*) files
- Metadata files:
  - `modules.dep`
  - `modules.alias`
  - `modules.symbols`
  - `modules.builtin`
- Each file has a corresponding `.bin` version, which is an optimized version of the corresponding text file



Module dependencies: *modules.dep*

- Some kernel modules can depend on other modules, based on the symbols (functions and data structures) that they use.
- Example: the `ubifs` module depends on the `ubi` and `mtd` modules.
  - `mtd` and `ubi` need to be loaded before `ubifs`
- These dependencies are described both in `/lib/modules/<kernel-version>/modules.dep` and in `/lib/modules/<kernel-version>/modules.dep.bin`
- Will be used by module loading tools.

Module alias: *modules.alias*

Kernel compiling

```
static const struct usb_device_id products [] = {
{
        // Linksys USB200M
        USB_DEVICE (0x077b, 0x2226),
        .driver_info =  (unsigned long) &ax8817x_info,
}, {
        // Netgear FA120
        USB_DEVICE (0x0846, 0x1040),
        .driver_info =  (unsigned long) &netgear_fa120_info,
},
...
        { },            // END
};
MODULE_DEVICE_TABLE(usb, products);
```

drivers/net/usb/asix_devices.c

The device driver source code lists which devices it supports

make modules

Module file

asix.ko

Containing the list of supported devices (module metadata)

make modules_install (depmod)

```
alias usb:v077Bp2226d*dc*dsc*dp*ic*isc*ip*in* asix
alias usb:v0846p1040d*dc*dsc*dp*ic*isc*ip*in* asix
....
```

modules.alias

A new USB device is plugged-in

Thanks to aliases, the asix module is loaded

udev has a rule for when MODALIAS is set

System operation

The USB controller driver reads USB device attributes: vendor id, product id, etc

The USB bus driver sends a MODALIAS string encoding these attributes to the udev process

MODALIAS=usb:v077bp2226d0001dcFFdscFFdp00icFFiscFFip00in00

```
ENV{MODALIAS}=="?*", RUN{builtin}+="kmod load $env{MODALIAS}"
....
```

/lib/udev/rules.d/80-drivers.rules

kmod load usb:v077bp2226d0001dcFFdscFFdp00icFFiscFFip00in00



Module utilities: *modinfo*

- `modinfo <module_name>`, for modules in `/lib/modules`
- `modinfo /path/to/module.ko`

```
# modinfo usb_storage
filename:       /lib/modules/5.18.13-200.fc36.x86_64/kernel/drivers/usb/storage/usb-storage.ko.xz
license:        GPL
description:    USB Mass Storage driver for Linux
author:         Matthew Dharm <mdharm-usb@one-eyed-alien.net>
alias:          usb:v*p*d*dc*dsc*dp*ic08isc06ip50in*
alias:          usb:v*p*d*dc*dsc*dp*ic08isc05ip50in*
alias:          usb:v*p*d*dc*dsc*dp*ic08isc04ip50in*
[...]
intree:         Y
name:           usb_storage
[...]
parm:           option_zero_cd:ZeroCD mode (1=Force Modem (default), 2=Allow CD-Rom (uint)
parm:           swi_tru_install:TRU-Install mode (1=Full Logic (def), 2=Force CD-Rom, 3=Force Modem) (uint)
parm:           delay_use:seconds to delay before using a new device (uint)
parm:           quirks:supplemental list of device IDs and their quirks (string)
```



Module utilities: *lsmod*

- Lists currently loaded kernel modules
- Includes
  - The reference count: incremented when the module is used by another module or by a user-space process, prevents from unloading modules that are in-use
  - Dependent modules: modules that depend on us
- Information retrieved through `/proc/modules`

```
$ lsmod
Module                  Size  Used by
tun                    61440  2
tls                   118784  0
rfcomm                 90112  4
snd_seq_dummy          16384  0
snd_hrtimer            16384  1
wireguard              94208  0
curve25519_x86_64      36864  1 wireguard
libcurve25519_generic  49152  2 curve25519_x86_64,wireguard
ip6_udp_tunnel         16384  1 wireguard
```



Module utilities: *insmod* and *rmmod*

- Basic tools to:
  - *load* a module: `insmod`
  - *unload* a module: `rmmod`
- Basic because:
  - Need a full path to the module `.ko` file
  - Do not handle module dependencies

```
# insmod /lib/modules/`uname -r`/kernel/fs/fuse/cuse.ko.xz
# rmmod cuse
```



Module utilities: *modprobe*

- *modprobe* is the more advanced tool for loading/unloading modules
- Takes just a module name as argument: `modprobe <module-name>`
- Takes care of dependencies automatically, using the `modules.dep` file
- Supports removing modules using `modprobe -r`, including its no longer used dependencies

```
# modinfo fat_test | grep depends
depends:        kunit,fat
# lsmod | grep -E "^(kunit|fat|fat_test)"
fat                    86016  1 vfat
# modprobe fat_test
# lsmod | grep -E "^(kunit|fat|fat_test)"
fat_test               24576  0
kunit                  36864  1 fat_test
fat                    86016  2 fat_test,vfat
# sudo modprobe -r fat_test
# lsmod | grep -E "^(kunit|fat|fat_test)"
fat                    86016  1 vfat
```



Passing parameters to modules

- Some modules have parameters to adjust their behavior
- Mostly for debugging/tweaking, as parameters are global to the module, not per-device managed by the module
- Through `insmod` or `modprobe`:

```
insmod ./usb-storage.ko delay_use=0
modprobe usb-storage delay_use=0
```

- `modprobe` supports configuration files: `/etc/modprobe.conf` or in any file in `/etc/modprobe.d/`:

```
options usb-storage delay_use=0
```

- Through the kernel command line, when the module is built statically into the kernel:

```
usb-storage.delay_use=0
```

  - `usb-storage` is the *module name*
  - `delay_use` is the *module parameter name*. It specifies a delay before accessing a USB storage device (useful for rotating devices).
  - `0` is the *module parameter value*



Modules in *sysfs*

- All modules are visible in *sysfs*, under `/sys/module/<name>`
- Lots of information available about each module
- For example, the `/sys/module/<name>/parameters` directory contains one file per module parameter
- Can read the current value of module parameters
- Some of them can even be changed at runtime (determined by the module code)
- Example: `echo 0 > /sys/module/usb_storage/parameters/delay_use`



Describing non-discoverable hardware: Device Tree



Describing non-discoverable hardware

1. Directly in the **OS/bootloader code**

- Using compiled data structures, typically in C
- How it was done on most embedded platforms in Linux, U-Boot.
- Considered not maintainable/sustainable on ARM32, which motivated the move to another solution.



Describing non-discoverable hardware

2. Using **ACPI** tables

- On *x86* systems, but also on a subset of ARM64 platforms
- Tables provided by the firmware



Describing non-discoverable hardware

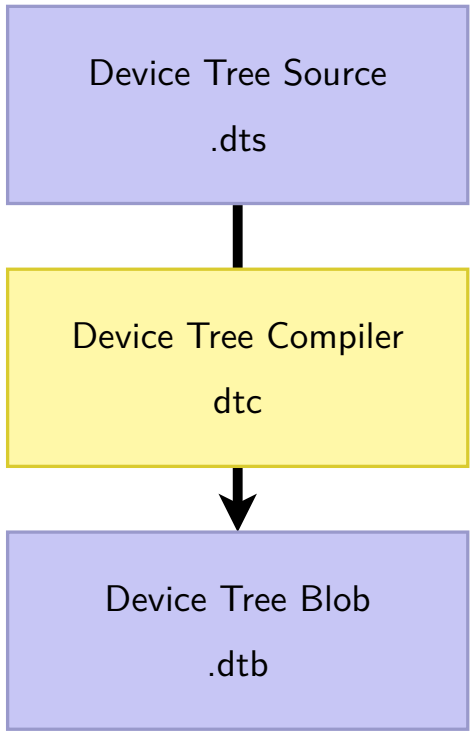
3. Using a **Device Tree**

- Originates from **OpenFirmware**, defined by Sun, used on SPARC and PowerPC
  - that's why many Linux/U-Boot functions related to DT have a `of_` prefix
- Now used by most embedded-oriented CPU architectures that run Linux: ARC, ARM64, RISC-V, ARM32, PowerPC, Xtensa, MIPS, etc.
- Writing/tweaking a DT is necessary when porting Linux to a new board, or when connecting additional peripherals



Device Tree: from source to blob

- A tree data structure describing the hardware is written by a developer in a **Device Tree Source** file, `.dts`
- Processed by the **Device Tree Compiler**, `dtc`
- Produces a more efficient representation: **Device Tree Blob**, `.dtb`
- Additional C preprocessor pass
- `.dtb` → accurately describes the hardware platform in an **OS-agnostic** way.
- `.dtb` ≈ few dozens of kilobytes
- DTB also called **FDT**, *Flattened Device Tree*, once loaded into memory.
  - `fdt` command in U-Boot
  - `fdt_` APIs





dtc example

```
$ cat foo.dts
/dts-v1/;

/ {
        welcome = <0xBADCAFE>;
        bootlin {
                webinar = "great";
                demo = <1>, <2>, <3>;
        };
};
```

```
$ dtc -I dts -O dtb -o foo.dtb foo.dts
$ ls -l foo.dt*
-rw-r--r-- 1 thomas thomas 169 ... foo.dtb
-rw-r--r-- 1 thomas thomas 102 ... foo.dts
```



dtc example

```
$ cat foo.dts
/dts-v1/;

/ {
        welcome = <0xBADCAFE>;
        bootlin {
                webinar = "great";
                demo = <1>, <2>, <3>;
        };
};
```

```
$ dtc -I dts -O dtb -o foo.dtb foo.dts
$ ls -l foo.dt*
-rw-r--r-- 1 thomas thomas 169 ... foo.dtb
-rw-r--r-- 1 thomas thomas 102 ... foo.dts
```

```
$ dtc -I dtb -O dts foo.dtb
/dts-v1/;

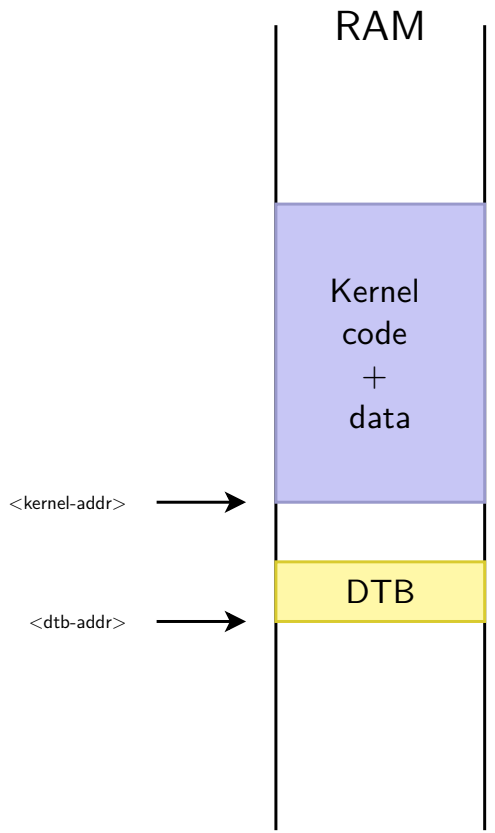
/ {
        welcome = <0xbadcafe>;

        bootlin {
                webinar = "great";
                demo = <0x01 0x02 0x03>;
        };
};
```



Device Tree: using the blob

- Can be **linked directly** inside a bootloader binary
  - For example: U-Boot, Barebox
- Can be **passed** to the operating system by the bootloader
  - Most common mechanism for the Linux kernel
  - U-Boot:
    `boot[z,i,m] <kernel-addr> - <dtb-addr>`
  - The bootloader can adjust the DTB before passing it to the kernel
- The DTB parsing can be done using `libfdt`, or ad-hoc code





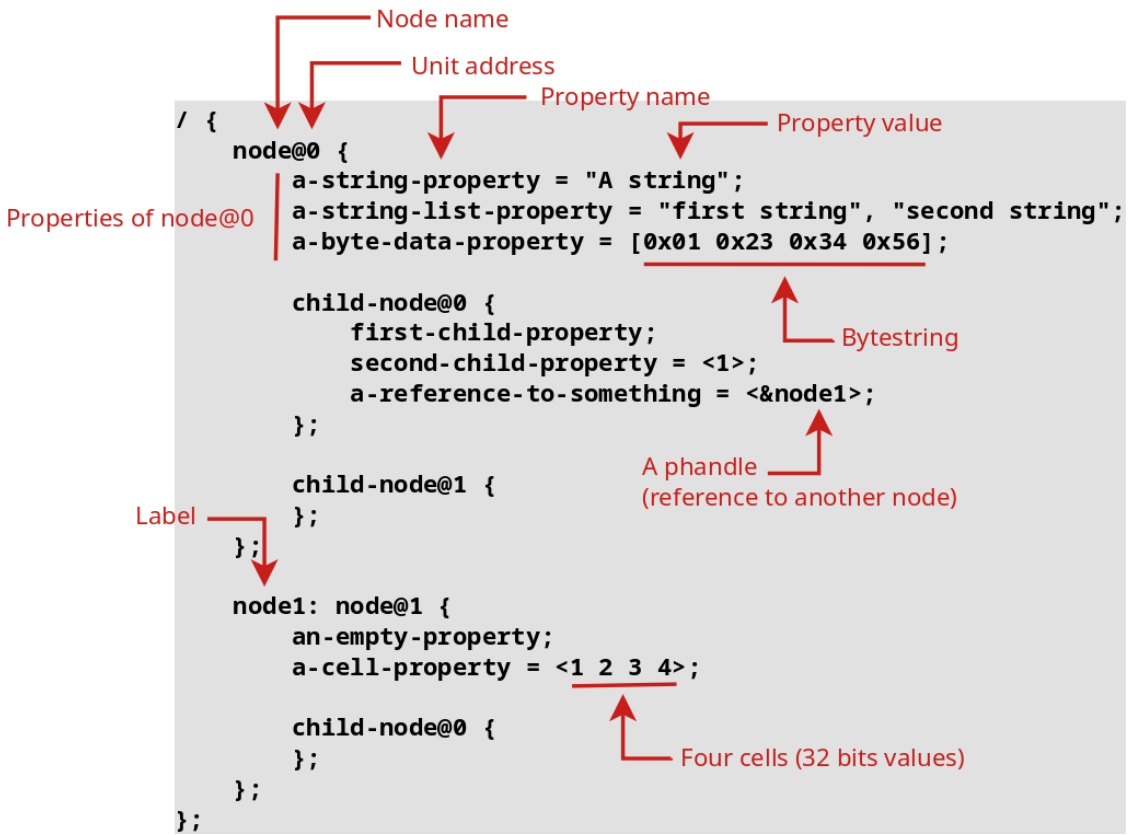
Where are Device Tree Sources located?

- Even though they are OS-agnostic, **no central and OS-neutral** place to host Device Tree sources and share them between projects
  - Often discussed, never done
- In practice, the Linux kernel sources can be considered as the **canonical location** for Device Tree Source files
  - `arch/<ARCH>/boot/dts/<vendor>/`
  - `arch/arm/boot/dts` (on ARM 32 architecture before Linux 6.5)
  - ≈ 4500 Device Tree Source files (`.dts` and `.dtsi`) in Linux as of 6.0.
- Duplicated/synced in various projects
  - U-Boot, Barebox, TF-A



Device Tree base syntax

- Tree of **nodes**
- Nodes with **properties**
- Node ≈ a device or IP block
- Properties ≈ device characteristics
- Notion of **cells** in property values
- Notion of **phandle** to point to other nodes
- `dtc` only does syntax checking, no semantic validation




```
/ {
    node@0 {
        a-string-property = "A string";
        a-string-list-property = "first string", "second string";
        a-byte-data-property = [0x01 0x23 0x34 0x56];

        child-node@0 {
            first-child-property;
            second-child-property = <1>;
            a-reference-to-something = <&node1>;
        };

        child-node@1 {
        };
    };

    node1: node@1 {
        an-empty-property;
        a-cell-property = <1 2 3 4>;

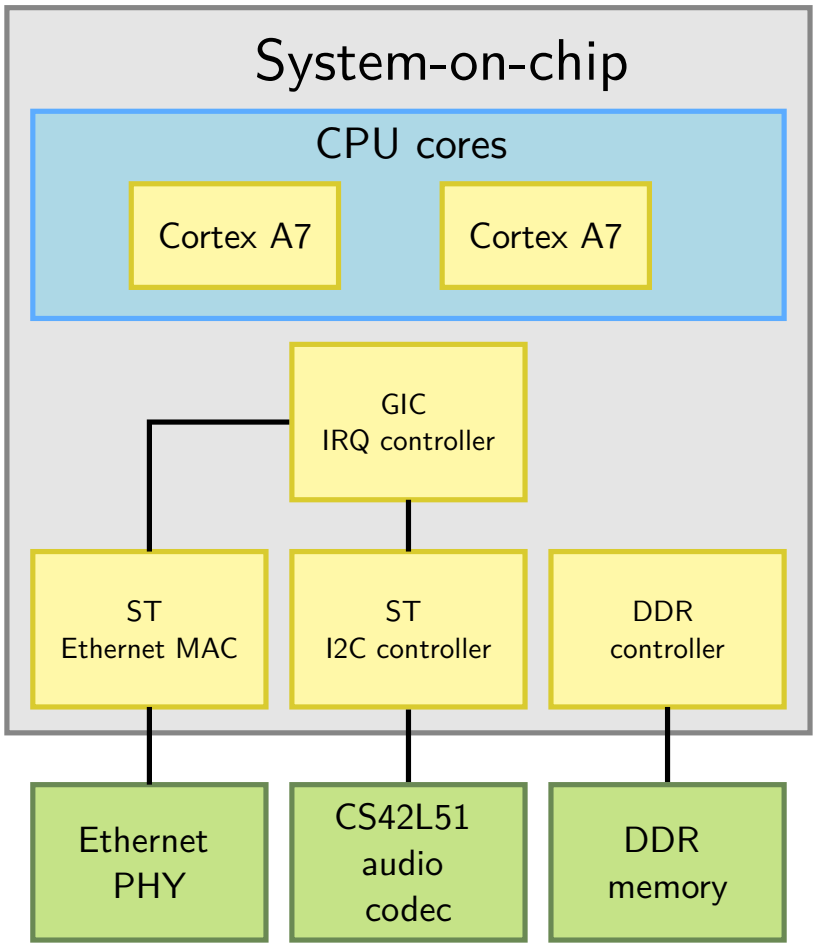
        child-node@0 {
        };
    };
};
```



DT overall structure: simplified example

```
/ {
  #address-cells = <1>;
  #size-cells = <1>;
  model = "STMicroelectronics STM32MP157C-DK2 Discovery Board";
  compatible = "st,stm32mp157c-dk2", "st,stm32mp157";

  cpus { ... };
  memory@0 { ... };
  chosen { ... };
  intc: interrupt-controller@a0021000 { ... };
  soc {
    i2c1: i2c@40012000 { ... };
    ethernet0: ethernet@5800a000 { ... };
  };
};
```



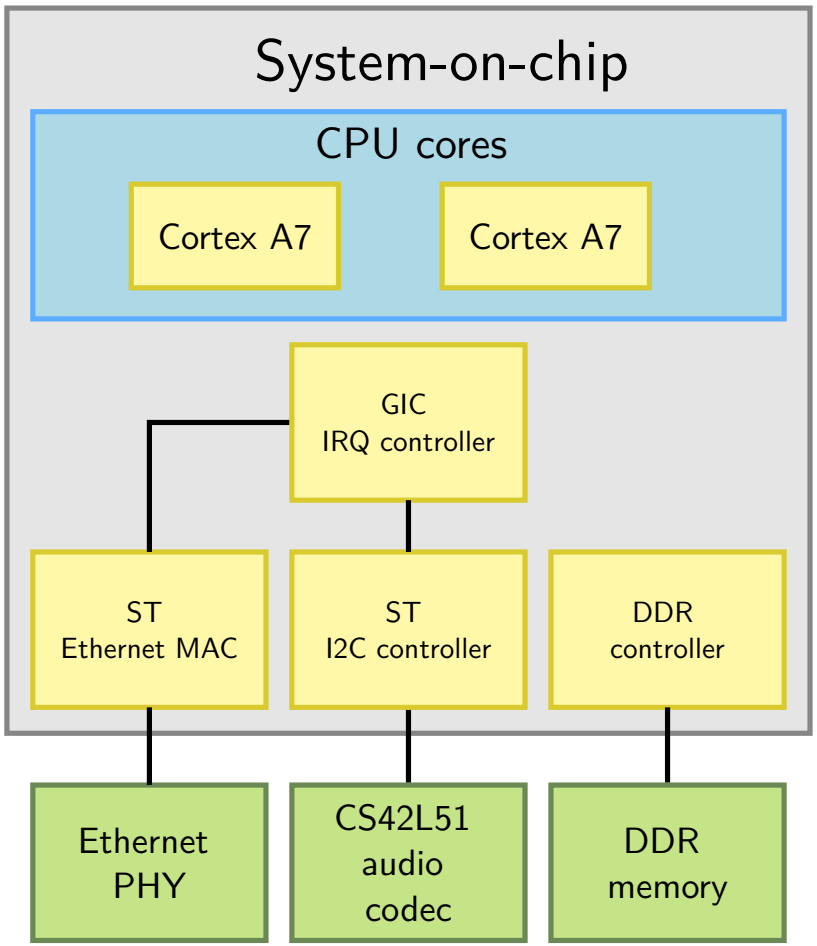


DT overall structure: simplified example

```
/ {
  cpus {
    #address-cells = <1>;
    #size-cells = <0>;
    cpu0: cpu@0 {
      compatible = "arm,cortex-a7";
      clock-frequency = <650000000>;
      device_type = "cpu";
      reg = <0>;
    };

    cpu1: cpu@1 {
      compatible = "arm,cortex-a7";
      clock-frequency = <650000000>;
      device_type = "cpu";
      reg = <1>;
    };
  };

  memory@0 { ... };
  chosen { ... };
  intc: interrupt-controller@a0021000 { ... };
  soc {
    i2c1: i2c@40012000 { ... };
    ethernet0: ethernet@5800a000 { ... };
  };
};
```

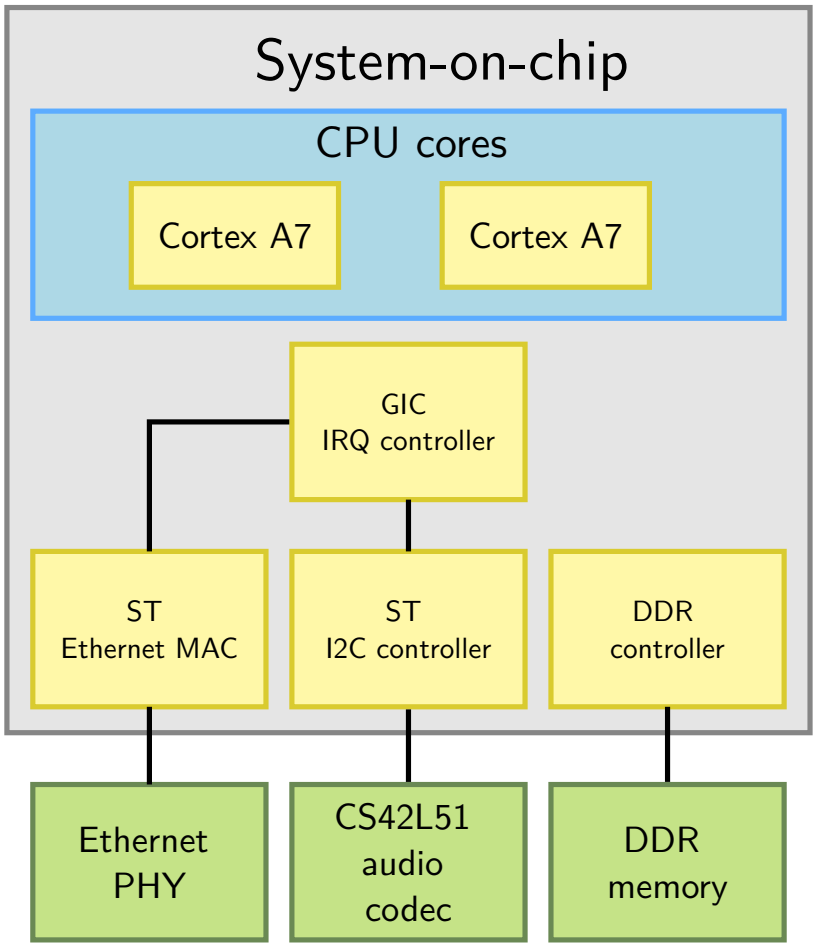




DT overall structure: simplified example

```
/ {
  cpus { ... };
  memory@0 {
    device_type = "memory";
    reg = <0x0 0x20000000>;
  };

  chosen {
    bootargs = "";
    stdout-path = "serial0:115200n8";
  };
  intc: interrupt-controller@a0021000 { ... };
  soc {
    i2c1: i2c@40012000 { ... };
    ethernet0: ethernet@5800a000 { ... };
  };
};
```





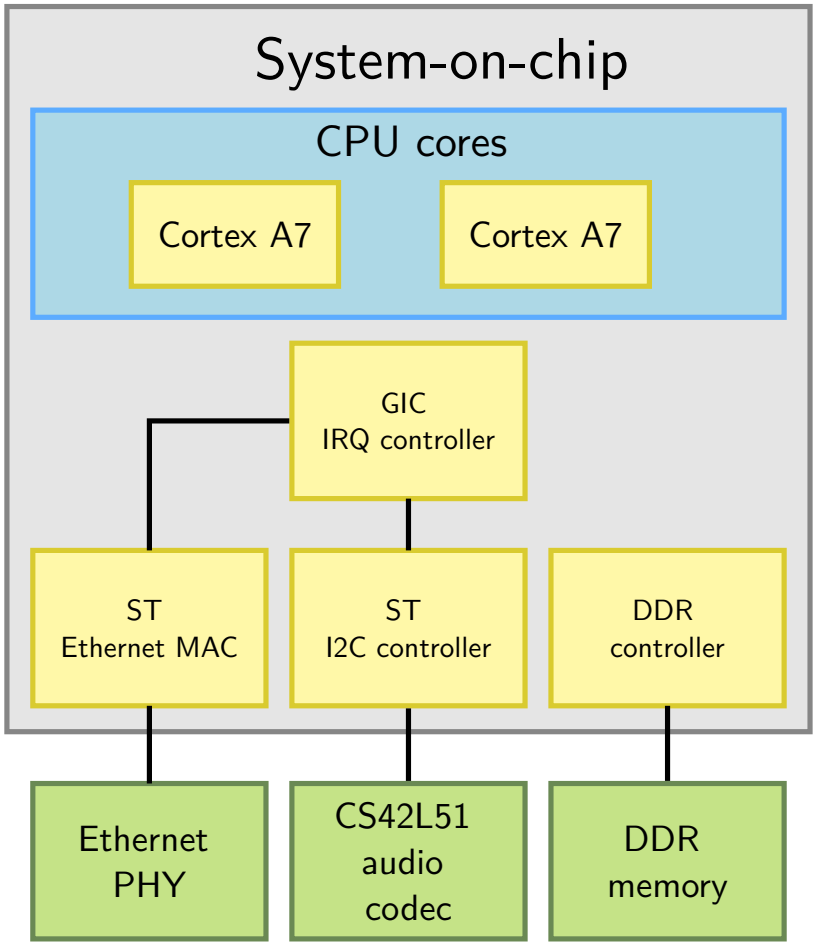
DT overall structure: simplified example

```
/ {
  cpus { ... };
  memory@0 { ... };
  chosen { ... };

  intc: interrupt-controller@a0021000 {
    compatible = "arm,cortex-a7-gic";
    #interrupt-cells = <3>;
    interrupt-controller;
    reg = <0xa0021000 0x1000>,
          <0xa0022000 0x2000>;
  };

  soc {
    compatible = "simple-bus";
    #address-cells = <1>;
    #size-cells = <1>;
    interrupt-parent = <&intc>;

    i2c1: i2c@40012000 { ... };
    ethernet0: ethernet@5800a000 { ... };
  };
};
```

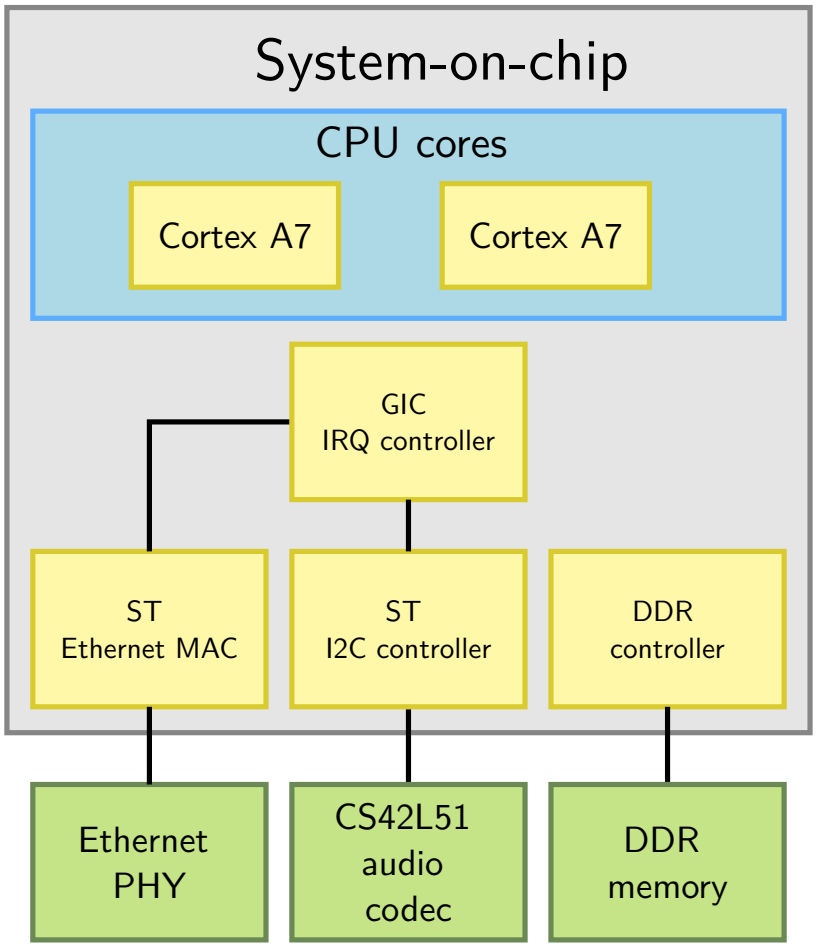




DT overall structure: simplified example

```
/ {
  cpus { ... };
  memory@0 { ... };
  chosen { ... };
  intc: interrupt-controller@a0021000 { ... };
  soc {
    i2c1: i2c@40012000 {
      compatible = "st,stm32mp15-i2c";
      reg = <0x40012000 0x400>;
      interrupts = <GIC_SPI 31 IRQ_TYPE_LEVEL_HIGH>,
                   <GIC_SPI 32 IRQ_TYPE_LEVEL_HIGH>;
      #address-cells = <1>;
      #size-cells = <0>;
      status = "okay";

      cs42l51: cs42l51@4a {
        compatible = "cirrus,cs42l51";
        reg = <0x4a>;
        reset-gpios = <&gpiog 9 GPIO_ACTIVE_LOW>;
        status = "okay";
      };
    };
    ethernet0: ethernet@5800a000 { ... };
  };
};
```



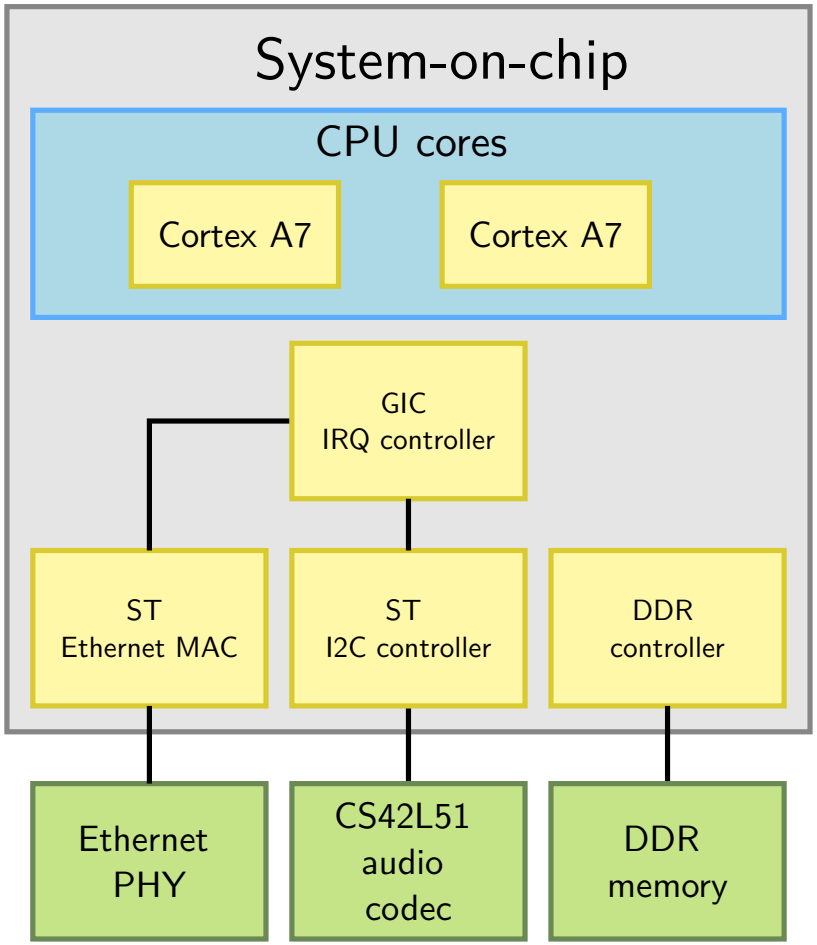


DT overall structure: simplified example

```
/ {
  cpus { ... };
  memory@0 { ... };
  chosen { ... };
  intc: interrupt-controller@a0021000 { ... };
  soc {
    compatible = "simple-bus";
    ...
    interrupt-parent = <&intc>;
    i2c1: i2c@40012000 { ... };

    ethernet0: ethernet@5800a000 {
      compatible = "st,stm32mp1-dwmac", "snps,dwmac-4.20a";
      reg = <0x5800a000 0x2000>;
      interrupts-extended = <&intc GIC_SPI 61 IRQ_TYPE_LEVEL_HIGH>;
      status = "okay";

      mdio0 {
        #address-cells = <1>;
        #size-cells = <0>;
        compatible = "snps,dwmac-mdio";
        phy0: ethernet-phy@0 {
          reg = <0>;
        };
      };
    };
  };
};
```





Device Tree inheritance

- Device Tree files are not monolithic, they can be split in several files, including each other.
- `.dtsi` files are included files, while `.dts` files are *final* Device Trees
  - Only `.dts` files are accepted as input to `dtc`
- Typically, `.dtsi` will contain
  - definitions of SoC-level information
  - definitions common to several boards
- The `.dts` file contains the board-level information
- The inclusion works by **overlaying** the tree of the including file over the tree of the included file, according to the order of the `#include` directives.
- Allows an including file to **override** values specified by an included file
- Uses the C pre-processor `#include` directive



Device Tree inheritance example

Definition of the STM32MP157A SoC

```
/ {
  soc {
    i2c1: i2c@40012000 {
      compatible = "st,stm32mp15-i2c";
      reg = <0x40012000 0x400>;
      status = "disabled";
    };
  };
};
```

stm32mp157.dtsi

+

Definition of the STM32MP157A-DK1 board

```
#include "stm32mp157.dtsi"

/ {
soc {
  i2c1: i2c@40012000 {
    pinctrl-names = "default";
    pinctrl-0 = <&i2c1_pins_a>;
    status = "okay";

      cs42l51: cs42l51@4a {
        compatible = "cirrus,cs42l51";
        reg = <0x4a>;
      };
    };
  };
};
```

stm32mp157a-dk1.dts

=

Compiled DTB

```
/ {
  soc {
    i2c1: i2c@40012000 {
      compatible = "st,stm32mp15-i2c";
      reg = <0x40012000 0x400>;
      pinctrl-names = "default";
      pinctrl-0 = <&i2c1_pins_a>;
      status = "okay";

      cs42l51: cs42l51@4a {
        compatible = "cirrus,cs42l51";
        reg = <0x4a>;
      };
    };
  };
};
```

stm32mp157a-dk1.dtb

Note: The real DTB is in binary format. Here we show the text equivalent of the DTB contents.

Note: The actual Device Trees for this platform are more complicated. This example is highly simplified.



Inheritance and labels

Doing:

soc.dtsi

```
/ {
  soc {
    usart1: serial@5c000000 {
      compatible = "st,stm32h7-uart";
      reg = <0x5c000000 0x400>;
      status = "disabled";
    };
  };
};
```

board.dts

```
#include "soc.dtsi"

/ {
  soc {
    serial@5c000000 {
      status = "okay";
    };
  };
};
```



Inheritance and labels

Doing:

soc.dtsi

```
/ {
  soc {
    usart1: serial@5c000000 {
      compatible = "st,stm32h7-uart";
      reg = <0x5c000000 0x400>;
      status = "disabled";
    };
  };
};
```

board.dts

```
#include "soc.dtsi"

/ {
  soc {
    serial@5c000000 {
      status = "okay";
    };
  };
};
```

Is exactly equivalent to:

soc.dtsi

```
/ {
  soc {
    usart1: serial@5c000000 {
      compatible = "st,stm32h7-uart";
      reg = <0x5c000000 0x400>;
      status = "disabled";
    };
  };
};
```

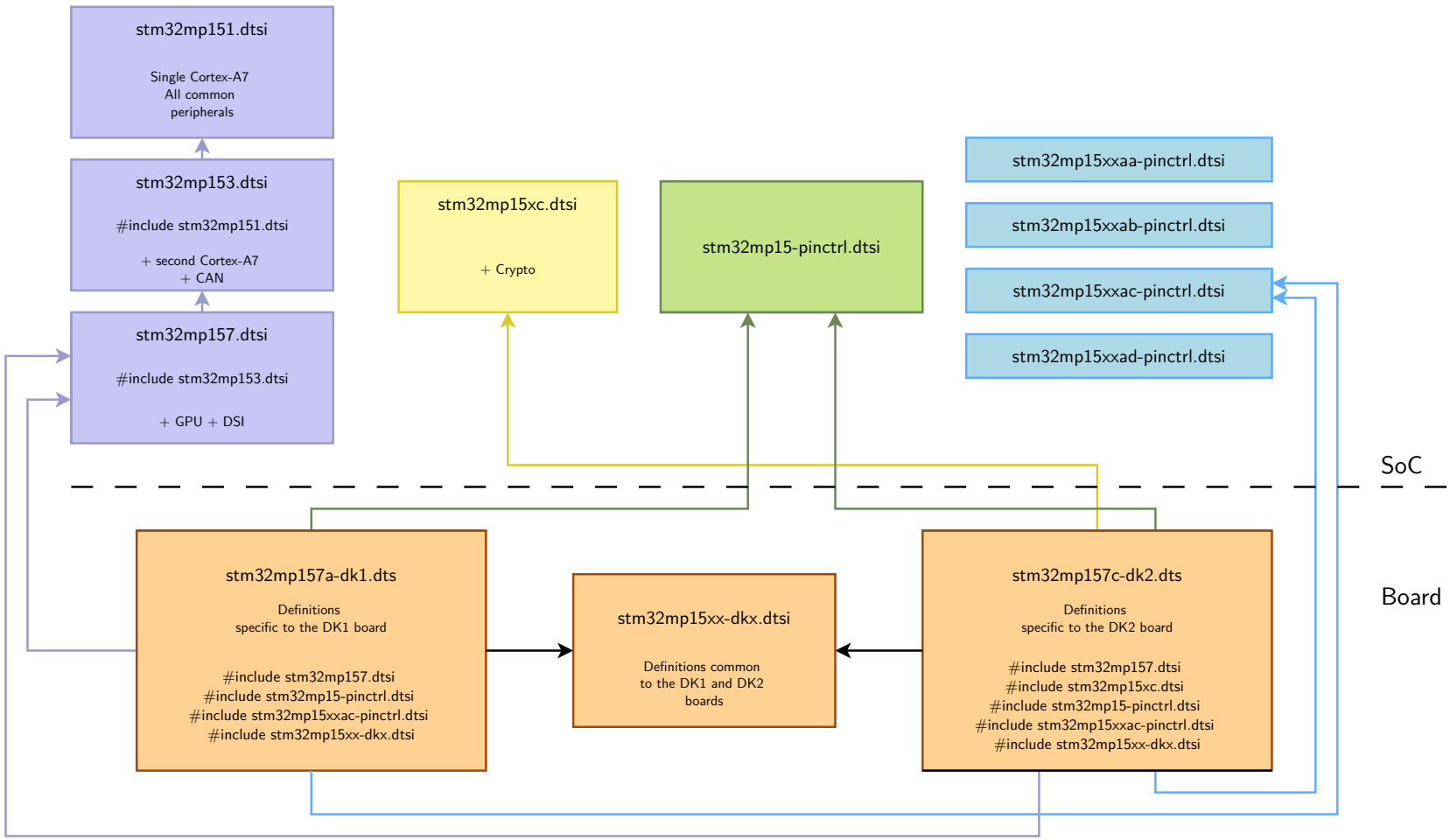
board.dts

```
#include "soc.dtsi"

&usart1 {
  status = "okay";
};
```

→ this solution is now often preferred

DT inheritance in STM32MP1 support





Device Tree design principles

- **Describe hardware** (how the hardware is), not configuration (how I choose to use the hardware)
- **OS-agnostic**
  - For a given piece of HW, Device Tree should be the same for U-Boot, FreeBSD or Linux
  - There should be no need to change the Device Tree when updating the OS
- Describe **integration of hardware components**, not the internals of hardware components
  - The details of how a specific device/IP block is working is handled by code in device drivers
  - The Device Tree describes how the device/IP block is connected/integrated with the rest of the system: IRQ lines, DMA channels, clocks, reset lines, etc.
- Like all beautiful design principles, these principles are sometimes violated.



Device Tree specifications

- How to write the correct nodes/properties to describe a given hardware platform ?
- **Device Tree Specifications** → base Device Tree syntax + number of standard properties.
  - https://www.devicetree.org/specifications/
  - Not sufficient to describe the wide variety of hardware.
- **Device Tree Bindings** → documents that each specify how a piece of HW should be described
  - Documentation/devicetree/bindings/ in Linux kernel sources
  - Reviewed by DT bindings maintainer team
  - Legacy: human readable documents
  - New norm: YAML-written specifications



Devicetree Specification

Release v0.3

devicetree.org

13 February 2020



Device Tree binding: old style

Documentation/devicetree/bindings/mtd/spear_smi.txt

This IP is *not* used on STM32MP1.

```
* SPEAr SMI

Required properties:
- compatible : "st,spear600-smi"
- reg : Address range of the mtd chip
- #address-cells, #size-cells : Must be present if the device has
sub-nodes
  representing partitions.
- interrupts: Should contain the STMMAC interrupts
- clock-rate : Functional clock rate of SMI in Hz

Optional properties:
- st,smi-fast-mode : Flash supports read in fast mode
```

```
Example:

        smi: flash@fc000000 {
                compatible = "st,spear600-smi";
                #address-cells = <1>;
                #size-cells = <1>;
                reg = <0xfc000000 0x1000>;
                interrupt-parent = <&vic1>;
                interrupts = <12>;
                clock-rate = <50000000>;        /* 50MHz */

                flash@f8000000 {
                        st,smi-fast-mode;
                        ...
                };
        };
```



Device Tree binding: YAML style

Documentation/devicetree/bindings/i2c/st,stm32-i2c.yaml

```
# SPDX-License-Identifier: (GPL-2.0-only OR BSD-2-Clause)
%YAML 1.2
---
$ id: http://devicetree.org/schemas/i2c/st,stm32-i2c.yaml#
$ schema: http://devicetree.org/meta-schemas/core.yaml#

title: I2C controller embedded in STMicroelectronics STM32 I2C platform

maintainers:
  - Pierre-Yves MORDRET <pierre-yves.mordret@st.com>

properties:
  compatible:
    enum:
      - st,stm32f4-i2c
      - st,stm32f7-i2c
      - st,stm32mp15-i2c

  reg:
    maxItems: 1

  interrupts:
    items:
      - description: interrupt ID for I2C event
      - description: interrupt ID for I2C error

  resets:
    maxItems: 1
```

```
  clocks:
    maxItems: 1

  dmas:
    items:
      - description: RX DMA Channel phandle
      - description: TX DMA Channel phandle

  ...

  clock-frequency:
    description: Desired I2C bus clock frequency in Hz. If not specified,
                 the default 100 kHz frequency will be used.
                 For STM32F7, STM32H7 and STM32MP1 SoCs, if timing
                 parameters match, the bus clock frequency can be from
                 1Hz to 1MHz.
    default: 100000
    minimum: 1
    maximum: 1000000

required:
  - compatible
  - reg
  - interrupts
  - resets
  - clocks
```



Device Tree binding: YAML style example

```
examples:
  - |
    //Example 3 (with st,stm32mp15-i2c compatible on stm32mp)
    #include <dt-bindings/interrupt-controller/arm-gic.h>
    #include <dt-bindings/clock/stm32mp1-clks.h>
    #include <dt-bindings/reset/stm32mp1-resets.h>
      i2c@40013000 {
          compatible = "st,stm32mp15-i2c";
          #address-cells = <1>;
          #size-cells = <0>;
          reg = <0x40013000 0x400>;
          interrupts = <GIC_SPI 33 IRQ_TYPE_LEVEL_HIGH>,
                       <GIC_SPI 34 IRQ_TYPE_LEVEL_HIGH>;
          clocks = <&rcc I2C2_K>;
          resets = <&rcc I2C2_R>;
          i2c-scl-rising-time-ns = <185>;
          i2c-scl-falling-time-ns = <20>;
          st,syscfg-fmp = <&syscfg 0x4 0x2>;
      };
```



Validating Device Tree in Linux

- `dtc` only does syntactic validation
- YAML bindings allow to do semantic validation
- Linux kernel `make` rules:
  - `make dt_binding_check` verify that YAML bindings are valid
  - `make dtbs_check` validate DTs currently enabled against YAML bindings
  - `make DT_SCHEMA_FILES=path/to/specific/yaml dtbs_check` validate DTs against a specific YAML binding



The `compatible` property

- Is a list of strings
  - From the most specific to the least specific
- Describes the specific **binding** to which the node complies.
- It uniquely identifies the **programming model** of the device.
- Practically speaking, it is used by the operating system to find the **appropriate driver** for this device.
- When describing real hardware, the typical form is `vendor,model`
- Examples:
  - `compatible = "arm,armv7-timer";`
  - `compatible = "st,stm32mp1-dwmac", "snps,dwmac-4.20a";`
  - `compatible = "regulator-fixed";`
  - `compatible = "gpio-keys";`
- Special value: `simple-bus` → bus where all sub-nodes are memory-mapped devices



`compatible` property and Linux kernel drivers

- Linux identifies as **platform devices**:
  - Top-level DT nodes with a `compatible` string
  - Sub-nodes of `simple-bus`
    - Instantiated automatically at boot time
- Sub-nodes of I2C controllers → *I2C devices*
- Sub-nodes of SPI controllers → *SPI devices*
- Each Linux driver has a table of compatible strings it supports
  - `struct of_device_id[]`
- When a DT node compatible string matches a given driver, the device is *bound* to that driver.

```
/ {
    timer {                                  → Platform device
        compatible = "...";
    };
    soc {
        compatible = "simple-bus";
        uart@1000 {                          → Platform device
            compatible = "...";
        };
        i2c@2000 {                           → Platform device
            compatible = "...";
            eeprom@65 {                      → I2C device
                compatible = "...";
            };
        };
    };
};
```



Matching with drivers in Linux: platform driver

drivers/tty/serial/stm32-usart.c

```
static const struct of_device_id stm32_match[] = {
        { .compatible = "st,stm32-uart", .data = &stm32f4_info},
        { .compatible = "st,stm32f7-uart", .data = &stm32f7_info},
        { .compatible = "st,stm32h7-uart", .data = &stm32h7_info},
        {},
}; MODULE_DEVICE_TABLE(of, stm32_match);

...

static struct platform_driver stm32_serial_driver = {
        .probe          = stm32_serial_probe,
        .remove         = stm32_serial_remove,
        .driver = {
                .name   = DRIVER_NAME,
                .pm     = &stm32_serial_pm_ops,
                .of_match_table = of_match_ptr(stm32_match),
        },
};
```



Matching with drivers in Linux: I2C driver

sound/soc/codecs/cs42l51.c

```
const struct of_device_id cs42l51_of_match[] = {
        { .compatible = "cirrus,cs42l51", },
        { }
}; MODULE_DEVICE_TABLE(of, cs42l51_of_match);
```

sound/soc/codecs/cs42l51-i2c.c

```
static struct i2c_driver cs42l51_i2c_driver = {
        .driver = {
                .name = "cs42l51",
                .of_match_table = cs42l51_of_match,
                .pm = &cs42l51_pm_ops,
        },
        .probe = cs42l51_i2c_probe,
        .remove = cs42l51_i2c_remove,
        .id_table = cs42l51_i2c_id,
};
```



reg property

- Most important property after `compatible`
- **Memory-mapped** devices: base physical address and size of the memory-mapped registers. Can have several entries for multiple register areas.

```
sai4: sai@50027000 {
    reg = <0x50027000 0x4>, <0x500273f0 0x10>;
};
```



reg property

- Most important property after `compatible`
- **Memory-mapped** devices: base physical address and size of the memory-mapped registers. Can have several entries for multiple register areas.
- **I2C** devices: address of the device on the I2C bus.

```
&i2c1 {
    hdmi-transmitter@39 {
        reg = <0x39>;
    };
    cs42l51: cs42l51@4a {
        reg = <0x4a>;
    };
};
```



reg property

- Most important property after `compatible`
- **Memory-mapped** devices: base physical address and size of the memory-mapped registers. Can have several entries for multiple register areas.
- **I2C** devices: address of the device on the I2C bus.
- **SPI** devices: chip select number

```
&qspi {
        flash0: mx66l51235l@0 {
                reg = <0>;
        };
        flash1: mx66l51235l@1 {
                reg = <1>;
        };
};
```



reg property

- Most important property after `compatible`
- **Memory-mapped** devices: base physical address and size of the memory-mapped registers. Can have several entries for multiple register areas.
- **I2C** devices: address of the device on the I2C bus.
- **SPI** devices: chip select number
- The unit address must be the address of the first `reg` entry.

```
sai4: sai@50027000 {
    reg = <0x50027000 0x4>, <0x500273f0 0x10>;
};
```



Status property

- The `status` property indicates if the device is really in use or not
  - `okay` or `ok` → the device is really in use
  - any other value, by convention `disabled` → the device is not in use
- In Linux, controls if a device is instantiated
- In `.dtsi` files describing SoCs: all devices that interface to the outside world have: `status = "disabled";`
- Enabled on a per-device basis in the board `.dts`



Resources: interrupts, clocks, DMA, reset lines, ...

- Common pattern for resources shared by multiple hardware blocks
  - Interrupt lines
  - Clock controllers
  - DMA controllers
  - Reset controllers
  - ...
- A Device Tree node describing the *controller* as a device
- References from other nodes that use resources provided by this *controller*

```
intc: interrupt-controller@a0021000 {
   compatible = "arm,cortex-a7-gic";
   #interrupt-cells = <3>;
   interrupt-controller;
   reg = <0xa0021000 0x1000>, <0xa0022000 0x2000>;
};

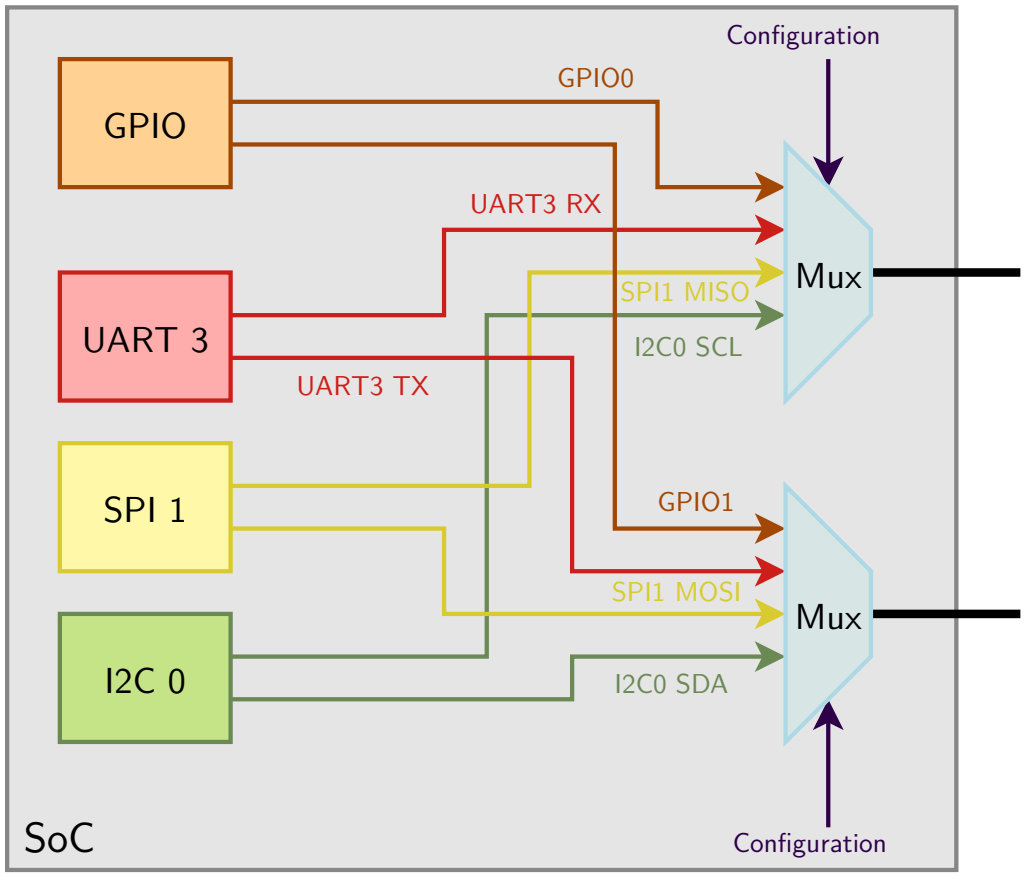
rcc: rcc@50000000 {
   compatible = "st,stm32mp1-rcc", "syscon";
   reg = <0x50000000 0x1000>;
   #clock-cells = <1>;
   #reset-cells = <1>;
};

dmamux1: dma-router@48002000 {
   compatible = "st,stm32h7-dmamux";
   reg = <0x48002000 0x1c>;
   #dma-cells = <3>;
   clocks = <&rcc DMAMUX>;
   resets = <&rcc DMAMUX_R>;
};

spi3: spi@4000c000 {
   interrupts = <GIC_SPI 51 IRQ_TYPE_LEVEL_HIGH>;
   clocks = <&rcc SPI3_K>;
   resets = <&rcc SPI3_R>;
   dmas = <&dmamux1 61 0x400 0x05>,  <&dmamux1 62 0x400 0x05>;
};
```

Pin-muxing description

- Most modern SoCs, including the STM32MP1, have more features than they have pins to expose those features to the outside world.
- Pins are muxed: a given pin can be used for one function **or** another
- A specific IP block in the SoC controls the muxing of pins: the **pinmux controller**
- The Device Tree describes which pin configurations are possible, and which configurations are used by the different devices.





Pin-muxing controllers on STM32MP1

arch/arm/boot/dts/st/stm32mp151.dtsi

```
pinctrl: pin-controller@50002000 {
        compatible = "st,stm32mp157-pinctrl";
        ...
};

pinctrl_z: pin-controller-z@54004000 {
        compatible = "st,stm32mp157-z-pinctrl";
        ...
};
```



Pin-muxing configuration

arch/arm/boot/dts/st/stm32mp15-pinctrl.dtsi

```
&pinctrl {
        ...
        i2c1_pins_a: i2c1-0 {
                pins {
                        pinmux = <STM32_PINMUX('D', 12, AF5)>, /* I2C1_SCL */
                                 <STM32_PINMUX('F', 15, AF5)>; /* I2C1_SDA */
                        bias-disable;
                        drive-open-drain;
                        slew-rate = <0>;
                };
        };
        ...
        m_can1_pins_a: m-can1-0 {
                pins1 {
                        pinmux = <STM32_PINMUX('H', 13, AF9)>; /* CAN1_TX */
                        slew-rate = <1>;
                        drive-push-pull;
                        bias-disable;
                };
                pins2 {
                        pinmux = <STM32_PINMUX('I', 9, AF9)>; /* CAN1_RX */
                        bias-disable;
                };
        };
        ...
};
```



Pin-muxing configuration

Table 8. Alternate function AF0 to AF7[1] (continued)

Port		AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7
		HDP/SYS/RTC	TIM1/2/16/17/ LPTIM1/SYS/ RTC	SAI1/4/I2C6/ TIM3/4/5/12/ HDP/SYS	SAI4/I2C2/ TIM8/ LPTIM2/3/4/5/ DFSDM1 /SDMMC1	SAI4/ I2C1/2/3/4/5/ USART1/ TIM15/LPTIM2/ DFSDM1/CEC	SPI1/I2S1/ SPI2/I2S2/ SPI3/I2S3/ SPI4/5/6/I2C1/ SDMMC1/3/ CEC	SPI3/I2S3/ SAI1/3/4/ I2C4/UART4/ DFSDM1	SPI2/I2S2/ SPI3/I2S3/ SPI6/ USART1/2/3/6/ UART7/ SDMMC2
Port D	PD6	-	TIM16_CH1N	SAI1_D1	DFSDM1_ CKIN4	DFSDM1_ DATIN1	SPI3_MOSI/ I2S3_SDO	SAI1_SD_A	USART2_RX
	PD7	TRACED6	-	-	DFSDM1_ DATIN4	I2C2_SCL	-	DFSDM1_ CKIN1	USART2_CK
	PD8	-	-	-	DFSDM1_ CKIN3	-	-	SAI3_SCK_B	USART3_TX
	PD9	-	-	-	DFSDM1_ DATIN3	-	-	SAI3_SD_B	USART3_RX
	PD10	RTC_REFIN	TIM16_BKIN	-	DFSDM1_ CKOUT	I2C5_SMBA	SPI3_MISO/ I2S3_SDI	SAI3_FS_B	USART3_CK
	PD11	-	-	-	LPTIM2_IN2	I2C4_SMBA	I2C1_SMBA	-	USART3_CTS/ USART3_NSS
	PD12	-	LPTIM1_IN1	TIM4_CH1	LPTIM2_IN1	I2C4_SCL	I2C1_SCL	-	USART3_RTS/ USART3_DE
	PD13	-	LPTIM1_OUT	TIM4_CH2	-	I2C4_SDA	I2C1_SDA	I2S3_MCK	-
	PD14	-	-	TIM4_CH3	-	-	-	SAI3_MCLK_B	-
	PD15	-	-	TIM4_CH4	-	-	-	SAI3_MCLK_A	-
Port E	PE0	-	LPTIM1_ETR	TIM4_ETR	-	LPTIM2_ETR	SPI3_SCK/ I2S3_CK	SAI4_MCLK_B	-
	PE1	-	LPTIM1_IN2	-	-	-	I2S2_MCK	SAI3_SD_B	-
	PE2	TRACECLK	-	SAI1_CK1	-	I2C4_SCL	SPI4_SCK	SAI1_MCLK_A	-
	PE3	TRACED0	-	-	-	TIM15_BKIN	-	SAI1_SD_B	-

Source: STM32MP157C datasheet. Note that `I2C1_SDA` is also available on pin `PF15` (not shown here).



Pin-muxing consumer

```
&i2c1 {
        pinctrl-names = "default", "sleep";
        pinctrl-0 = <&i2c1_pins_a>;
        pinctrl-1 = <&i2c1_sleep_pins_a>;
        ...
};
```

- Typically board-specific, in `.dts`
- `pinctrl-0`, `pinctrl-1`, `pinctrl-X` provides the pin mux configurations for the different **states**
- `pinctrl-names` gives a name to each state, mandatory even if only one state
- States are mutually exclusive
- The driver is responsible for switching between states
- `default` state is automatically set up when the device is *probed*



Example: LED and I2C device

- Let's see how to describe an LED and an I2C device connected to the DK1 platform.
- Create `arch/arm/boot/dts/st/stm32mp157a-dk1-custom.dts` which includes `stm32mp157a-dk1.dts`

```
#include "stm32mp157a-dk1.dts"
```

- Make sure `stm32mp157a-dk1-custom.dts` gets compiled to a DTB by changing arch/arm/boot/dts/Makefile

```
dtb-$(CONFIG_ARCH_STM32) +=
        ...
        stm32mp157a-dk1.dtb
        stm32mp157a-dk1-custom.dtb
```

- `make dtbs`

```
  DTC     arch/arm/boot/dts/st/stm32mp157a-dk1-custom.dtb
```



Example: describe an LED

stm32mp157a-dk1-custom.dts

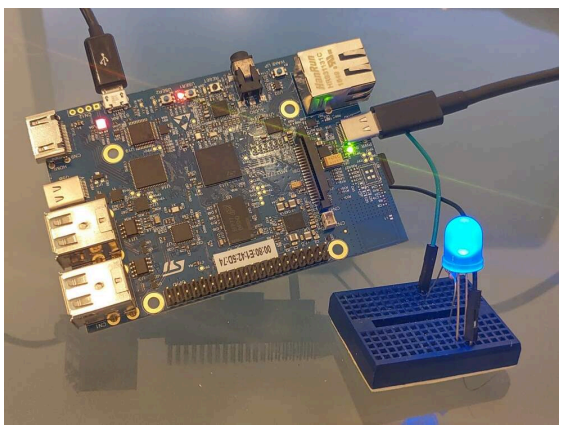
```
#include "stm32mp157a-dk1.dts"

/ {
        leds {
                compatible = "gpio-leds";
                webinar {
                        label = "webinar";
                        gpios = <&gpioe 1 GPIO_ACTIVE_HIGH>;
                };
        };
};
```

shell

```
# echo 255 > /sys/class/leds/webinar/brightness
```

1	ARD_D0	PE7	USART7_RX
2	ARD_D1	PE8	USART7_TX
3	ARD_D2	PE1	IO
4	ARD_D3	PD14	TIM4_CH3
5	ARD_D4	PE10	IO
6	ARD_D5	PD15	TIM4_CH4
7	ARD_D6	PE9	TIM1_CH1
8	ARD_D7	PD1	IO



Example: connect I2C temperature, humidity and pressure sensor

stm32mp157a-dk1-custom.dts

```
&i2c5 {
        status = "okay";
        clock-frequency = <100000>;
        pinctrl-names = "default", "sleep";
        pinctrl-0 = <&i2c5_pins_a>;
        pinctrl-1 = <&i2c5_pins_sleep_a>;

        pressure@76 {
                compatible = "bosch,bme280";
                reg = <0x76>;
        };
};
```

shell

```
# cat /sys/bus/iio/devices/iio:device2/
in_humidityrelative_input
49147
# cat /sys/bus/iio/devices/iio:device2/in_pressure_input
101.567167968
# cat /sys/bus/iio/devices/iio:device2/in_temp_input
24380
```

CN13	1	ARD_D8	PG3	IO
	2	ARD_D9	PH6	TIM12_CH1
	3	ARD_D10	PE11	SPI4_NSS and TIM1_CH2
	4	ARD_D11	PE14	SPI4_MOSI and TIM1_CH4
	5	ARD_D12	PE13	SPI4_MISO
	6	ARD_D13	PE12	SPI4_SCK
	7	GND	-	GND
	8	VREFP	-	VREF+
	9	ARD_D14	PA12	I2C5_SDA
	10	ARD_D15	PA11	I2C5_SCL

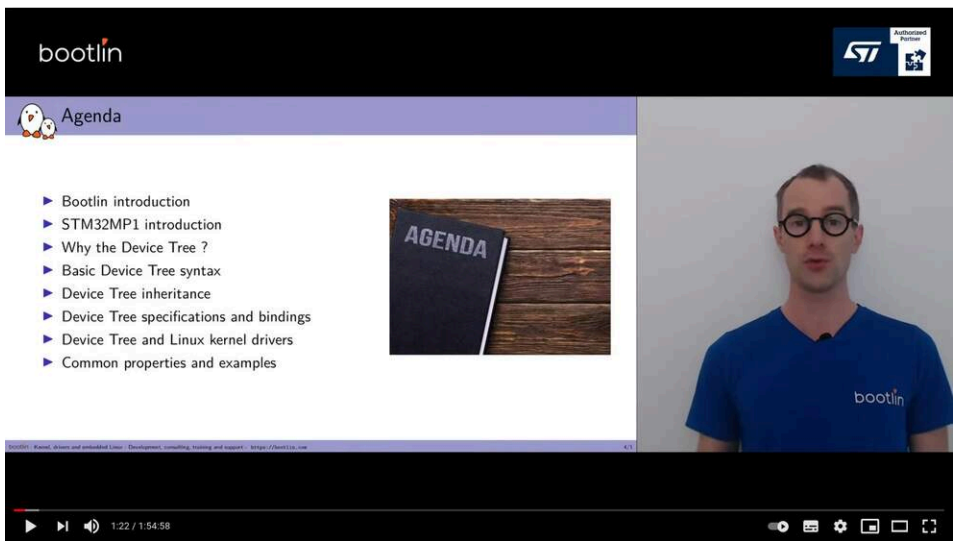


Details at https://bootlin.com/blog/building-a-linux-system-for-the-stm32mp1-connecting-an-i2c-sensor/

Further details about the Device Tree

Check out our *DeviceTree 101 webinar*, by Thomas Petazzoni (2021)

- Slides: https://bootlin.com/blog/device-tree-101-webinar-slides-and-videos/
- Video: https://youtu.be/a9CZ1Uk3OYQ





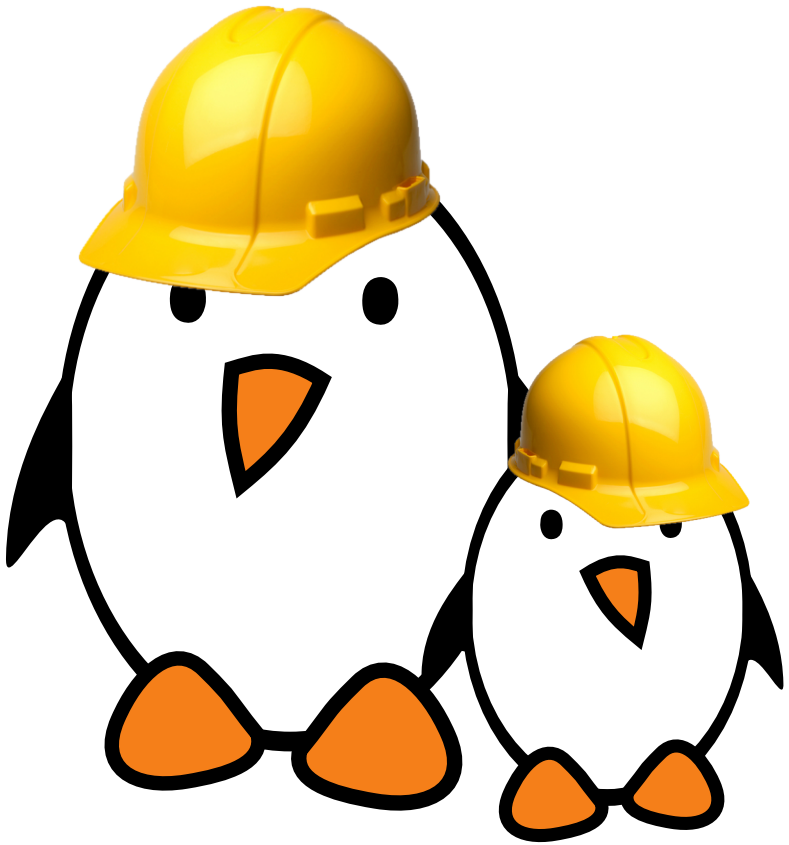
Discoverable hardware: USB and PCI



Discoverable hardware

- Some busses have built-in hardware discoverability mechanisms
- Most common busses: USB and PCI
- Hardware devices can be enumerated, and their characteristics retrieved with just a driver or the bus controller
- Useful Linux commands
  - `lsusb`, lists all USB devices detected
  - `lspci`, lists all PCI devices detected
  - A detected device does not mean it has a kernel driver associated to it!
- Association with kernel drivers done based on product ID/vendor ID, or some other characteristics of the device: device class, device sub-class, etc.

Practical lab - Accessing hardware devices



Time to start the practical lab!

- Exploring the contents of `/dev` and `/sys` and the devices available on the embedded hardware platform.
- Using GPIOs and LEDs.
- Modifying the Device Tree to control pin multiplexing and declare an I2C-connected joystick.
- Adding support for a USB audio card using Linux kernel modules
- Adding support for the I2C-connected joystick through an out-of-tree module.